Experiment:5

Aim: Define Linked List ADT and implement its operations.

Linked List ADT (Abstract Data Type)

Definition:
A Linked List is a linear data structure where elements (called nodes) are stored in memory, and each node contains:

1. Data – the value of the node
2. Next – a pointer/reference to the next node

Unlike arrays, linked lists do not store elements in contiguous memory. They are dynamic in size, and insertion/deletion is efficient at any position.

Types of Linked Lists:

- Singly Linked List: Each node points to the next node.
- Doubly Linked List: Each node points to both the next and previous node.
- Circular Linked List: Last node points back to the first node.

2. Operations on Linked List ADT

Basic Operations:

1. Create: Initialize an empty list.
2. Insert: Add a node at:
    - Beginning (head)
    - End (tail)
    - Specific position
3. Delete: Remove a node by:
    - Value
    - Position
4. Search: Find a node by value.

Dept of AIML

5. Display: Print all nodes.

3. Algorithm for Singly Linked List Operations

Insert at End:

1. Create a new node with the given data.
2. If the list is empty, set head = new node.
3. Else, traverse to the last node and set its next = new node.

Delete by Value:

1. Start from the head.
2. If head contains the value, move head to next.
3. Else, traverse to find the node whose next contains the value.
4. Update current node's next to skip the deleted node.

Search:

1. Start from the head.
2. Traverse each node, comparing data with target.
3. Return position if found; else return not found.

Display:

1. Start from the head.
2. Traverse all nodes and print data.

Program of Singly Linked List:

```
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Linked List class
class LinkedList:
    def __init__(self):
        self.head = None

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
```

Dept of AIML

```
        return
    last = self.head
    while last.next:
        last = last.next
    last.next = new_node

# Insert at beginning
def insert_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node

# Delete by value
def delete_node(self, key):
    temp = self.head
    if temp is not None:
        if temp.data == key:
            self.head = temp.next
            temp = None
            return
    prev = None
    while temp is not None and temp.data != key:
        prev = temp
        temp = temp.next
    if temp is None:
        print("Value not found")
        return
    prev.next = temp.next
    temp = None

# Search
def search(self, key):
    current = self.head
    position = 1
    while current:
        if current.data == key:
            return position
        current = current.next
        position += 1
    return -1  # Not found

# Display
def display(self):
    current = self.head
    while current:
        print(current.data, end=" -> ")
        current = current.next
```

Dept of AIML

```
        print("None")

# Example usage
ll = LinkedList()
ll.insert_end(10)
ll.insert_end(20)
ll.insert_beginning(5)
ll.insert_end(30)

print("Linked List:")
ll.display()

ll.delete_node(20)
print("After deleting 20:")
ll.display()

pos = ll.search(30)
print(f"30 found at position: {pos}")
```

OUTPUT:

```
Linked List:
5 -> 10 -> 20 -> 30 -> None
After deleting 20:
5 -> 10 -> 30 -> None
30 found at position: 3
```

Dept of AIML